

LV14 스택, 큐

<https://youtu.be/qATP7p87jUk?feature=shared>

Stack(스택)과 Queue(큐) 자료구조

Stack(스택)

스택이란?

 **STACK** 이라는 자료구조는 규칙이 다음과 같다

1. **저장**: 항상 위에만 저장한다
2. **읽기**: 항상 제일 위에 있는 데이터만 읽을 수 있다
3. **삭제**: 항상 제일 위에 있는 데이터만 삭제 할 수 있다

 **핵심 개념**: LIFO (Last In, First Out) - 나중에 들어간 것이 먼저 나온다

스택의 용도

- **STACK**은 위 규칙을 가진 전용자료구조 이고 배열을 이용해서 구현해도 좋고 또는 **LIST**를 이용해서 구현해도 좋다. 구현 방법은 개발자의 자유다.
- **실생활 예시**: 접시 쌓기, 책 쌓기, 되돌리기(Undo) 기능 등

링크드리스트 기반 스택 구현

```
#pragma once
#include <iostream>
#include <string>
#include <vector>

struct Node
{
    int data;
    Node* next;
```

```

};

Node* top = nullptr;    // 📌 TOP 스택의 최상단을 가리키는 포인터
Node* bottom = nullptr; // 📌 END 스택의 바닥을 가리키는 포인터

// 📌 Stack - 요소 추가
void push(int value)
{
    Node* buff = new Node(); // 📌 NEW 새 노드 생성
    buff->data = value;      // 📌 데이터 저장
    buff->next = top;        // 📌 기존 최상단을 다음으로 연결
    top = buff;              // 📌 TOP 새 노드를 최상단으로 설정
}

// 📌 Stack - 요소 제거
void pop()
{
    Node* backup = top;     // 📌 삭제할 노드 백업
    top = top->next;         // 📌 TOP 최상단을 다음 노드로 이동
    delete backup;          // 📌 메모리 해제
}

// 📌 Stack - 최상단 요소 확인
int Top()
{
    return top->data;        // 📌 최상단 데이터 반환
}

```

☁ 동작 과정:

```

push(1) → [1]
push(3) → [3] → [1]
push(2) → [2] → [3] → [1]
pop()  → [3] → [1]    // 2가 제거됨

```

12 34 배열 기반 스택 구현

```

#pragma once
#include <iostream>
#include <string>
#include <vector>

int mArr[256] = {};    // 📦 스택 저장 배열
int top = -1;          // 📌 현재 최상단 위치 (-1: 비어있음)

// 📦 Stack - 요소 추가
void push(int value)
{
    mArr[++top] = value; // 📈 top 증가 후 데이터 저장
}

// 📦 Stack - 요소 제거
void pop()
{
    if (top == -1)      // 🚫 스택이 비어있는지 확인
    {
        // 빈 스택에서 pop 시도
    }
    else
    {
        top--;          // 📉 top만 감소 (실제 데이터는 덮어쓰기됨)
    }
}

// 👁 Stack - 최상단 요소 확인
int Top()
{
    if (top == -1)      // 🚫 스택이 비어있는지 확인
    {
        std::cout << "Stack is empty" << std::endl;
        return -1;
    }
    else
    {
        std::cout << "Top value : " << mArr[top] << std::endl;
    }
}

```

```

    return mArr[top];
}
}

```

배열 스택 시각화:

초기 상태: mArr[256] = {}, top = -1

push(1): top = 0, mArr[0] = 1

push(3): top = 1, mArr[1] = 3

push(2): top = 2, mArr[2] = 2

push(5): top = 3, mArr[3] = 5

현재 상태:

Index: [0][1][2][3][4]...

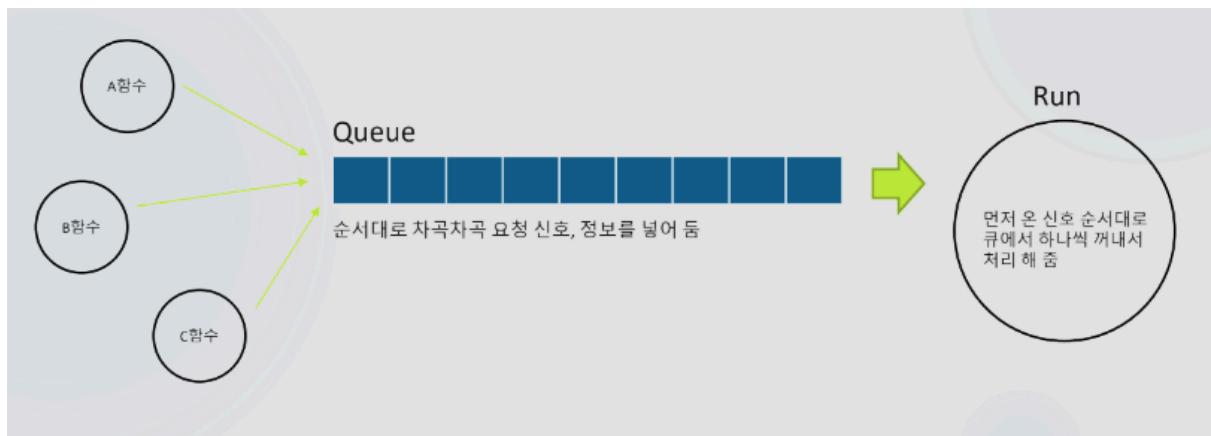
Value: [1][3][2][5][]...

↑

top

🚂 Queue(큐)

큐란?



📖 스택과 비슷한 자료구조이며 컵업에서 광장히 많이 사용된다

- 여러 함수에서 신호들이 한꺼번에 들어오면, 순차 처리를 위한 대기열 용도로 사용된다

큐의 특징

📄 큐와 스택의 차이

- 스택: **pop** 할때, 가장 마지막에 들어온 값을 가장 먼저 제거한다
- 큐: **pop** 할때, 가장 처음에 들어온 값을 가장 먼저 제거한다

💡 핵심: FIFO (First In, First Out) - 먼저 들어간 것이 먼저 나온다

큐의 시각적 표현

🎯 Queue 동작:

A함수 ————
| Queue: [□][□][□][□][□][□][□][□] ———→ Run
B함수 ———— | 순서대로 차곡차곡 요청 신호, 정보를 넣어 둠 (업무 중 신호 순
서대로
| 처리할 하나씩 꺼내서
C함수 ———— | 처리 해 줌)

🏗️ 링크드리스트 기반 큐 구현

```
#include <iostream>
#include <string>
#include <list>
#include <assert.h>
#include <vector>
#include <stack>
#include <queue>

struct Node
{
    int data;
    Node* next;
};

Node* head = nullptr; // 🚩 큐의 앞쪽 (제거되는 쪽)
Node* tail = nullptr; // ⬅️ END 큐의 뒤쪽 (추가되는 쪽)

// 📦 Queue - 요소 추가 (뒤쪽에 추가)
```

```

void push(int value)
{
    if (head == nullptr) // 🏠 첫 번째 요소인 경우
    {
        Node* buff = new Node();
        buff->data = value;
        head = buff;
        tail = buff;
    }
    else // 🔗 기존 요소가 있는 경우
    {
        tail->next = new Node(); // 🆕 새 노드를 tail 다음에 연결
        tail = tail->next; // ⬅️ tail을 새 노드로 이동
        tail->data = value; // 📝 데이터 저장
    }
}

// 🍷 Queue - 요소 제거 (앞쪽에서 제거)
void pop()
{
    Node* backup = head; // 📁 삭제할 노드 백업
    head = head->next; // 🚩 head를 다음 노드로 이동
    delete backup; // 🗑️ 메모리 해제
}

// 👁 Queue - 앞쪽 요소 확인
int top()
{
    return head->data; // 🔍 가장 앞쪽 데이터 반환
}

```

🎯 큐 동작 과정:

```

push(1): head → [1] ← tail
push(3): head → [1] → [3] ← tail
push(2): head → [1] → [3] → [2] ← tail
push(5): head → [1] → [3] → [2] → [5] ← tail

```

pop(): head→[3]→[2]→[5]←tail // 10이 제거됨 (FIFO)

Stack vs Queue 비교

구분	Stack	Queue
원리	LIFO (후입선출)	FIFO (선입선출)
추가위치	top (맨 위)	rear/tail (맨 뒤)
제거위치	top (맨 위)	front/head (맨 앞)
실생활 예시	접시 쌓기, Undo 기능	대기줄, 프린터 작업 대기열
주요 용도	함수 호출, 재귀, 괄호 검사	작업 스케줄링, BFS

구현 방식 선택

배열 기반 vs 링크드리스트 기반

배열 기반

- **장점:** 구현 간단, 빠른 접근
- **단점:** 크기 제한, 메모리 낭비 가능

링크드리스트 기반

- **장점:** 동적 크기, 메모리 효율적
- **단점:** 포인터 오버헤드, 구현 복잡

핵심 포인트

Stack의 핵심

- `push()` : 맨 위에 추가
- `pop()` : 맨 위에서 제거
- `top()` : 맨 위 요소 확인

Queue의 핵심

- `push()` : 맨 뒤에 추가
- `pop()` : 맨 앞에서 제거
- `front()` : 맨 앞 요소 확인

⚠ 주의사항

- 빈 자료구조에서 `pop()`, `top()` 호출 시 예외 처리 필요
- 동적 할당 시 메모리 해제 잊지 않기
- 인덱스 범위 체크 (배열 기반의 경우)

이 두 자료구조는 프로그래밍에서 매우 기본적이면서도 중요한 개념이므로, 구현 원리를 정확히 이해하고 활용할 수 있어야 합니다! 🚀

“강의는 많은데, 왜 나는 아직도 코드를 못 짤까?”

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 ‘**연습**’이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얄팍코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얄팍코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 **스스로 쪼개고 설계하는 힘**
- 다양한 조건을 만족시키는 **실제 구현 능력**
- 기능 단위가 아닌, **프로그램 단위로 사고하는 습관**
- 마침내 **자신의 힘으로 코드를 끝까지 작성하는 경험**

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얏얏코딩 상담방](#)

▼ oopy:hide



문제 1번

아래 소스코드는 힙에 노드를 4개 생성한 코드입니다.

3을 입력하면 3의 배수로 4개 노드를 생성해 주세요.



이제 while문을 이용해서 만들어진 모든 노드를 출력 해 주세요.

[힌트]

```
cin >> input;
for (int i = 1; i<=4; i++) {
    addNode(i * input);
}
```

입력 예제

3

출력 결과

3 6 9 12



문제 2번

숫자 하나를 입력하세요 (11 ~ 36 까지 숫자)

입력받은 번호에 해당하는 문자 부터 노드 4개만 연결시켜주세요.

11 : A

12 : B

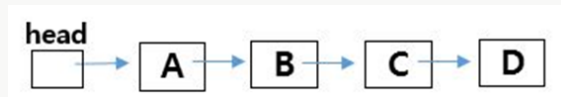
13 : C

14 : D

...

36 : Z

만약 11을 입력받았다면, 아래와 같이 구성하면 됩니다.



그리고 for문을 돌려 모든 노드를 출력 해 주세요.

입력 예제

11

출력 결과

ABCD

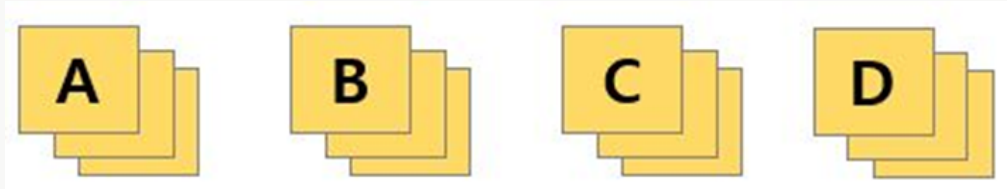


문제 3번

네 종류의 카드가 있습니다. (A,B,C,D)

몇 장을 뽑을지 입력받아주세요

n장을 뽑으려고 하는데 **중복 없이 뽑는 경우를 출력** 해주세요.



만약 2를 입력했다면

AB

AC

AD

BA

BC

...

DC

만약 3을 입력했다면

ABC

ABD

ACB

...

DCB

가지치기 힌트

via 전역배열을 이용해서 중복 판단 가능

입력 예제

2

출력 결과

AB

AC

AD

BA

BC

BD

CA

CB

CD

DA

DB

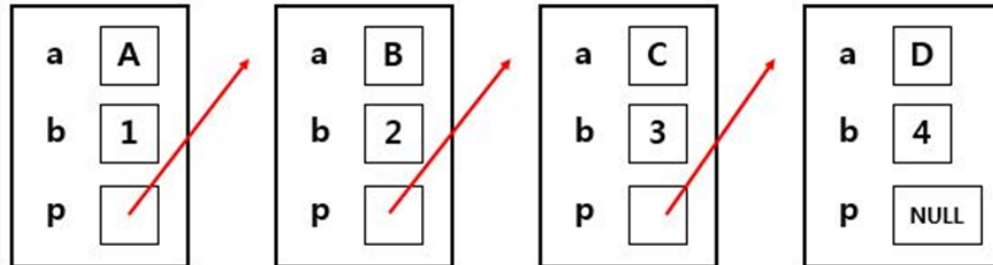
DC



문제 4번

숫자 n 을 입력 받고, n 개만큼 노드를 만들고 연결 해주세요.

들어가는 값은 순차적으로 A, B, C, D... / 1, 2, 3, 4... 입니다.



노드를 구성한 후

대문자는 **for문**으로 출력하고

숫자들은 **while**로 모두 출력 해 주세요.

ex1)

입력: 4

출력: A B C D

1 2 3 4

ex2)

입력: 3

출력: A B C

1 2 3

입력 예제

4

출력 결과

A B C D

1 2 3 4



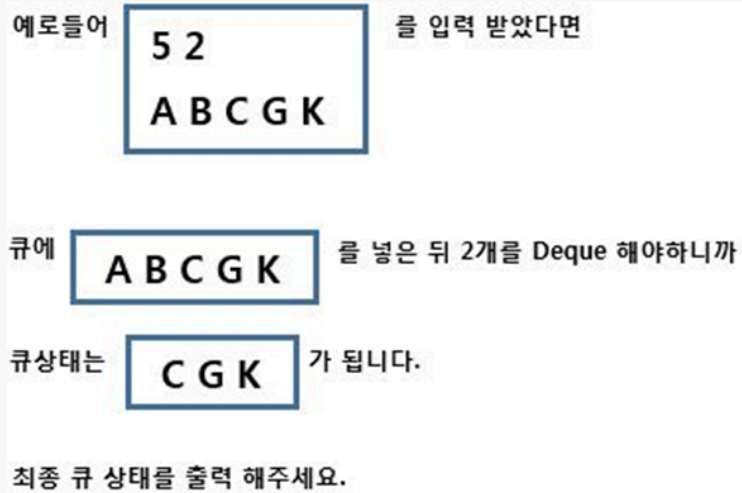
문제 5번

링크드리스트로 Queue를 만들어주세요.

입력의 첫번째 숫자는 Queue에 넣을 문자 갯수(Enqueue)

두번째 숫자는 Dequeue 할 갯수 입니다.

그 다음 문자들은 Queue에 넣을 문자들 입니다.



입력 예제

3 1

A B C

출력 결과

B C



문제 6번

숫자와 문자가 섞인 문장을 입력 받습니다.

이때 문장 속에서 숫자는 하나만 존재합니다.(최대 15글자)

ex1) ATP1326TTA

ex2) PPPT756

이런 숫자와 문자가 섞인 한문장에서 숫자만 파싱하여 5를 더한 숫자를 출력하세요.

아래 예제를 참고하세요.

ex) 1999POW

→ 1999 + 5 = 2004 출력

ex) ATP1326TTA

→ 1326 + 5 = 1331 출력

ex) PPPT756

→ 756 + 5 = 761 출력

[Tip]

parsing(파싱) : 문장에서 내가 원하는 값을 뽑아내는 처리를 뜻합니다.

입력 예제

1999POW

출력 결과

2004



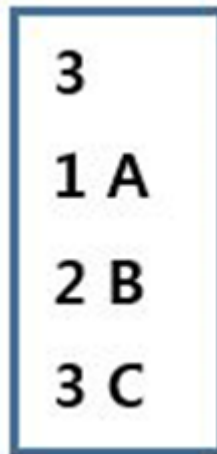
문제 7번

한 노드가 다음과 같은 구조체로 된 큐를 배열로 구현해 주세요.

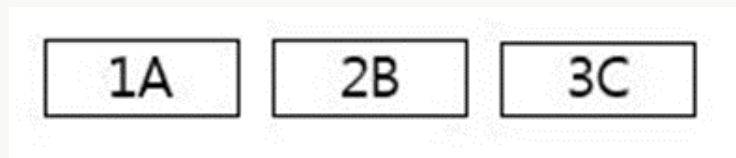


Node queue[10];

이제 입력 개수에 따라 입력이 주어 집니다.



이렇게 입력을 받았다면 큐에 아래와 같이 채워집니다.



모두 채운 후, 모든 값을 pop 해주세요.

이제 pop을 할때마다 나오는 값을 출력 해주세요.

입력 예제

4

1 A

2 B

3 C

4 D

출력 결과

1 A

2 B

3 C

4 D



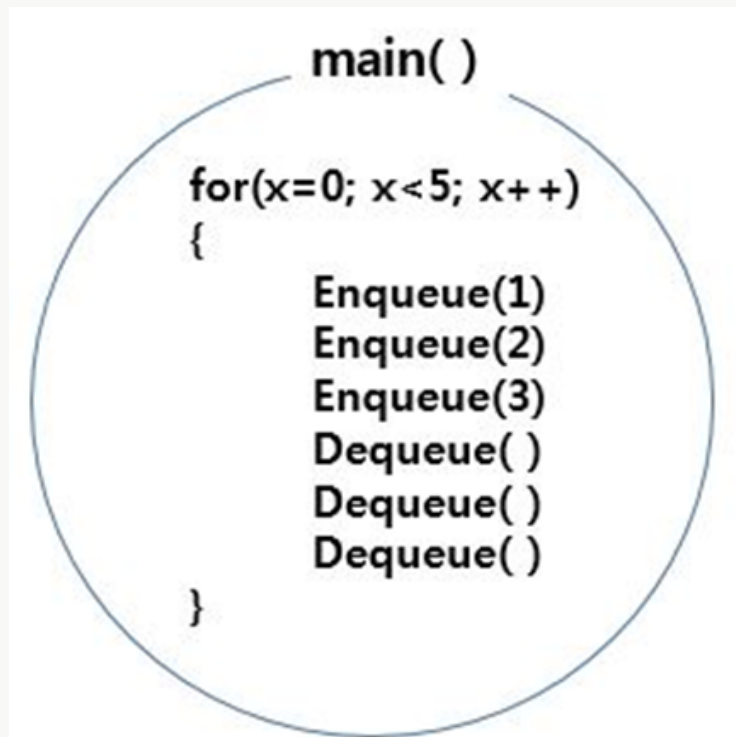
문제 8번

10칸짜리 큐를 만들어주세요.

숫자 하나를 입력받고, Enqueue 3회 / Dequeue 3회 반복하는 소스코드를 작성 해 주세요.

Dequeue할때마다 숫자를 출력하면 됩니다.

만약 n이 5라면 아래와 같은 **main** 함수가 동작되도록 구현 하시면 됩니다.



입력 예제

5

출력 결과

123123123123



문제 9번

다음 2차배열을 하드코딩 하주세요.

	A	B	C	D
0	3	5	1	4
1	2	2	1	1
2	0	1	2	3
3	3	1	3	1

문자로 된 숫자1개 또는 문자 1개를 입력 받으세요.

해당되는 한줄을 출력 하면 됩니다.

ex)

1을 입력 받으면 2211 출력

0을 입력 받으면 3514 출력

A를 입력 받으면 3203 출력

입력 예제

1

출력 결과

2211



문제 10번

꺅쇠 < 와 >, 숫자로 이루어진 한 문장을 입력 받으세요.

꺅쇠가 열리면 ('<') 이후에 꺅쇠가 닫혀야 합니다.('>')

정상적으로 꺅쇠가 열리고 닫혔는지 판단하는 프로그램을 작성하세요.

정상이면 "정상", 비정상이면 "비정상" 출력. (최대 20글자)

1. ex) <35<6<912>>10> ⇒ 정상

2. ex) 56<7>>65 ⇒ 비정상

3. ex) >>15<< ⇒ 비정상

4. ex) 612<>7<>91 ⇒ 정상

입력 예제

<35<6<912>>10>

출력 결과

정상



문제 11번

Text 한문장을 입력 받으세요.

그리고 **커서 위치**를 입력 받으세요.

예를 들어 ABCDEF와 커서위치 2를 입력 받았다면



CMD는 세가지 입니다.

L: Left Cursor

R: Right Cursor

D: Delete Cursor

이제 **CMD 4개를 입력받고** CMD에 따라 처리를 해주세요.

만약 **RRLD**를 입력 받았다면

오른쪽 2칸, 왼쪽 1칸 커서를 이동시키고, Delete를 한번 수행하면 되므로



와 같은 상태가 됩니다.

커서 위치는 3번 index 글자 앞에 있습니다.

명령어를 수행하고 커서가 있는 위치를 출력 해주세요.

ex)

ABCDEF
2
RRLD



[출력]

3

입력 예제

ABCDEF

2

RRLD

출력 결과

3



문제 12번

큐를 이용해서 풀어보세요.

척척박사님은 B, I, A, H 슈퍼영웅들 중 출동할 사람을 순서대로 뽑아야 합니다.

척척박사님은 항상 영웅B를 시작으로 다섯번째 사람을 선택합니다.

반복적으로 다섯번째 사람을 선택했을 때, 출동하는 영웅들의 순서를 출력 하세요.

B	I	A	H

B	I	A	H
①	②	③	④
⑤			

→ 첫번째 출동자는 배동맨

B	I	A	H
	①	②	③
	④	⑤	

→ 두번째 출동자는 캡틴A

B	I	A	H
			①
			③
			⑤
		②	
		④	

→ 세번째 출동자는 힐크오

출동순서 결과: B A H I

[힌트]

4번 (Dequeue
Enqueue 후에 1회 Dequeue하고 출력 하면
항상 다섯번째 사람이 출력 됩니다.

출력 결과

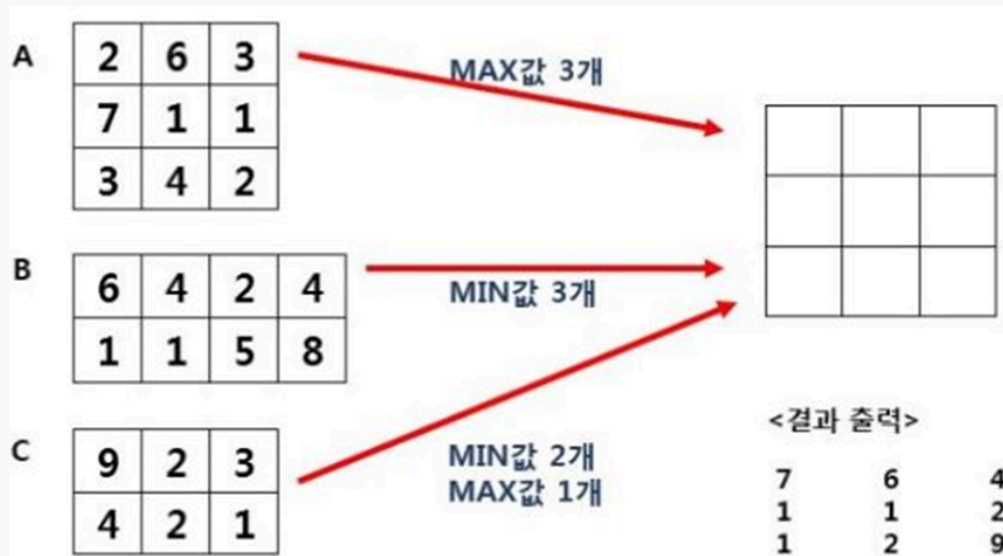
B A H I



문제 13번

A, B, C 배열에 있는 정예멤버를 선정하여 후보배열에 넣으려 합니다. (A, B, C 배열은 하드코딩)

- A배열에서 MAX 3명 선출. (가장 큰 수 3개)
- B배열에서 MIN 3명 선출. (가장 작은 수 3개)
- C배열에서 MIN 2명, MAX 1명 선출. (가장 작은 수 2개, 가장 큰 수 1개)



출력 결과

7 6 4

1 1 2

1 2 9